

Reloading a station export after a gauge reset

Operations note DS/OP/07 · Data services · Reviewed 14 January 2026

Symptoms

The station's daily total is short and the loader's own log says nothing is wrong. In the log the day looks like a clean run: the export was fetched, every block was read, and the summary line reports fewer readings than the station's counter did. The gap is always at the end of the day rather than the beginning, and it is always a whole number of blocks.

The other sign is a duplicate warning in the block log, with two entries carrying the same sequence number and different checksums. One warning is normal after a reset. A run of them means the reset happened part way through the day and everything after it is being rejected block by block.

Why the loader rejects the second block

Block sequence numbers come from the station and not from the loader. The loader keeps the highest number it has committed for the day and treats anything at or below it as a repeat, which is the right thing to do on a poor link: a station that retransmits sends the same block twice, and the loader must not count it twice.

A reset returns the counter to zero, so the blocks after it carry sequence numbers the loader has already committed. Nothing about them looks new and the loader discards them for exactly the reason it should. The reset time is the only thing that distinguishes a continuation from a repeat, and the loader does not know it unless it is told.

The fix

When a gauge is reset in the field the station's counter returns to zero, and the export for that day contains two blocks carrying the same sequence number. The loader accepts the first and rejects the second. That is the right behaviour for a duplicated transmission and the wrong behaviour here, and the day's record ends up short by however much arrived after the reset.

The fix is to requeue the affected blocks with the reset time supplied, so that the loader treats the second block as a continuation rather than a repeat. The function below is the one to call. It lives in `tarnbeck.stations.maintenance` and is safe to run more than once: a block that already carries a good checksum is skipped.

```
from tarnbeck.stations import Export, reload_window

def reload_after_reset(station, reset_at):
    export = Export.open(station)
    window = reload_window(reset_at, margin_minutes=45)

    for block in export.blocks(window):
        if block.checksum_ok():
            continue
        export.requeue(block, reason="gauge reset")

    return export.commit(dry_run=False)
```

Run it from the loader host and not from a workstation, and take the reset time from the field log rather than from the station clock, which is the thing that was wrong. The margin defaults to forty-five minutes either side of the reset; widen it only if the field log is vague about the time, because a wide window requeues blocks that were never in doubt.

If `commit` returns a count lower than the number of blocks in the window, stop there and raise it with data services before running anything again. A short count means blocks were rejected for a second reason, and requeuing them repeatedly will not find out what it was.

Running it

Run it from the loader host, in the maintenance environment, with the station code and the reset time as they appear in the field log:

```
$ ssh loader-01.tarnbeck.example
$ tarnbeck-maint reload-after-reset --station VQ1 \
  --reset-at '2026-01-29T09:15' --confirm
```

The command prints the window it will use, the number of blocks in it and the number it intends to requeue, and then waits. Read the three numbers before answering. If the window is wider than the field log justifies, stop and narrow it rather than letting it run.

Without `--confirm` the command does everything except commit, which is the safe way to see what it would do. The dry run takes a few seconds and there is no reason to skip it.

Afterwards

Check the day's totals against the gauge's own counter before closing the ticket. The loader reports the number of blocks committed and not the number of readings, and a day can come back with the right number of blocks and the wrong number of readings if a block was truncated in the field.

Note the reset in the station log, with the time taken from the field log and not the time the reload ran. The next person to look at a gap in the series will look there first, and a reload with nothing written beside it is indistinguishable from a gap nobody has dealt with.

If the count is short

A short count means blocks were rejected for a second reason and the reload has not found out what it was. Do not run it again: a second pass requeues the same blocks, reports the same count, and fills the log with attempts that bury the original failure.

Take the sequence numbers the command reports as rejected and look them up in the block log. A checksum failure on a block truncated in the field cannot be fixed by reloading and needs the station visited. A rejection with no reason recorded is a loader fault, and goes to data services with the run's log attached.

The block header, for reference

Each block in an export begins with a fixed header, which is what the loader reads to decide whether it has seen the block before:

```
SEQ=00184 STATION=VQ1 START=2026-01-29T08:45Z  
COUNT=0180 INTERVAL=15s CRC=8f2a41d6
```

SEQ is the station's counter and is the field a reset returns to zero. START is the station clock, which is not to be trusted after a reset - which is why the reset time comes from the field log and not from the export.

CRC covers the readings and not the header, so a block with a good checksum and a wrong sequence number is possible and is exactly what a reset produces. Nothing in the header says a reset happened.

Rolling back

A committed reload can be undone within the same day by reverting the station's day to the snapshot the loader takes before it commits. The snapshot is kept for seven days and is named for the run.

After seven days the only route back is a fresh fetch of the export from the station, which is possible while the station's own buffer still holds the day - about three weeks - and impossible afterwards. A reload that has committed and cannot be rolled back is an incident and is raised as one, whatever the size of the gap.